



Machine Learning (MLOps)

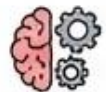
An Integrated MLOps System for GI Polyp Image Classification

Team No 16

USN	Name
1RV23AI126	ZAID SHARIEFF
1RV23AI130	ADITYA K
1RV23AI129	UNMESH RAJ
1RV23AI124	YASHPREET GOYAL



Agenda



Introduction



Motivation (Why this title)



Problem Definition



Objectives



Data set Description



Risk Analysis



Tools and Techniques used



Phase I : Machine Learning Model and its analysis



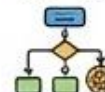
Phase II : Building MLOPs



Risk Analysis



Tools and Techniques used

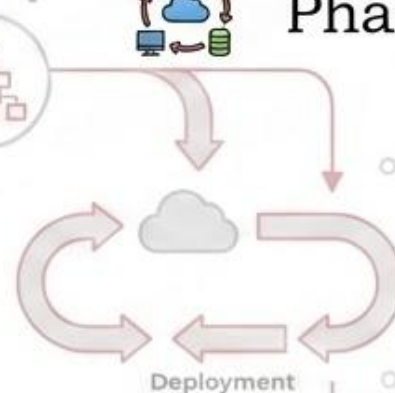


Phase I : Machine Learning Model

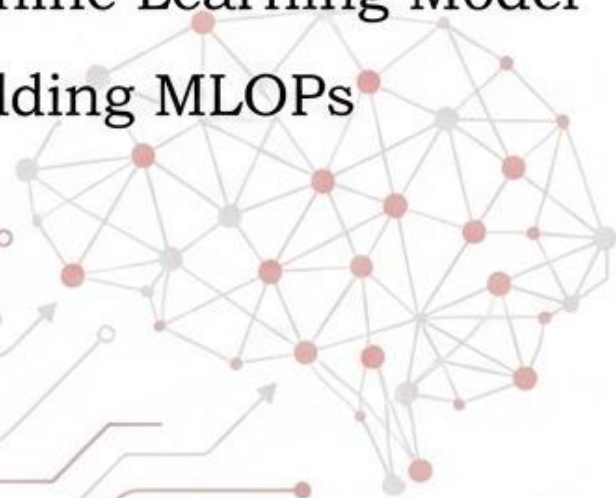


Phase II : Building MLOPs

MLOps



Deployment





Introduction

- This project focuses on building an **end-to-end MLOps pipeline** for **automatic classification of gastrointestinal polyps** using deep learning.
- As some of the polyps maybe harmless, many of them are cancerous and are fatal.



Motivation

- Early detection of gastrointestinal polyps is critical for **preventing colorectal cancer**.
- Manual diagnosis is **time-consuming and subjective**, motivating automation using deep learning.
- The title reflects the goal of **combining medical image classification with MLOps Tools to ease the workload of managing the data efficiently**.



Problem Definition

- Manual diagnosis of gastrointestinal polyps from endoscopic images is time-consuming and subject to human variability.



Objectives

- Build a deep learning model for **polyp classification**
- Track experiments and metrics using **MLflow**
- Version datasets and models using **DVC**
- Detect **data drift** using **Evidently AI**
- Visualize system metrics using **Grafana**
- Deploy the trained model using **TorchServe**
- Ensure **reproducibility and traceability** across the ML lifecycle



Dataset Description

Dataset used:

(Classification)

Source: Public medical endoscopy dataset

Contains labeled images of gastrointestinal conditions, including polyps

Dataset was:

- Split into **training and validation sets (70% training and 30% validation)**
- Preprocessed using resizing, normalization, and augmentation techniques
- Dataset versions were tracked using **DVC**



Dataset Description

Data Explorer

Version 1 (1.3 GB)

- ▼  kvasir-dataset
 - ▼  kvasir-dataset
 - ▶  dyed-lifted-polyps
 - ▶  dyed-resection-mar
 - ▶  esophagitis
 - ▶  normal-cecum
 - ▶  normal-pylorus
 - ▶  normal-z-line
 - ▶  polyps
 - ▶  ulcerative-colitis



Dataset Description

```
▼ data
  ▼ raw \ Kvasir-dataset
    ▼ kvasir-dataset
      > dyed-lifted-polyps
      > dyed-resection-margins
      > esophagitis
      > normal-cecum
      > normal-pylorus
      > normal-z-line
      > polyps
      > ulcerative-colitis
      > kvasir-dataset-split
    > uploaded
  ◆ .gitignore
  ≡ medical_images.db
  ≡ raw.dvc
```



Data Preprocessing

```
utils.py  X
src > ml > utils.py > ...
1  from torchvision import transforms
2  import sys
3  import os
4
5  # ☒ Fix Python path so that 'src' is recognized
6  sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '..', '..')))
7
8  def get_image_transform(img_size: int = 224, mode: str = "val"):
9      NORM_MEAN = [0.485, 0.456, 0.406]
10     NORM_STD = [0.229, 0.224, 0.225]
11
12     base = [
13         transforms.Resize((img_size, img_size)),
14         transforms.ToTensor(),
15         transforms.Normalize(NORM_MEAN, NORM_STD)
16     ]
17
18     if mode == "train":
19         return transforms.Compose([
20             transforms.RandomRotation(15),
21             transforms.RandomHorizontalFlip(),
22             transforms.RandomVerticalFlip(),
23             transforms.ColorJitter(0.1, 0.1),
24             transforms.RandomAffine(0, translate=(0.05, 0.05), scale=(0.95, 1.05)),
25             *base
26         ])
27
28     return transforms.Compose(base)
29
```



Risk Analysis

Data risks: Class imbalance and image variability

Model risks: Overfitting due to limited data

Operational risks: Model drift after deployment

Mitigation strategies:

- Transfer learning using **ResNet101**
- Validation monitoring
- Drift detection with **Evidently AI**
- Version control using **DVC**



Risk Analysis

Risk Matrix

		IMPACT			
		LOW	MED	HIGH	EXTREME
LIKELIHOOD	RARE	minor lib issue	False Positive	data poisoning	False Negative
	UNLIKELY	minor documentation gap	Data drift	critical system crash	model theft
	LIKELY	Regulatory non compliance	versioning conflict	sensitive data leaks	
	ALMOST CERTAIN	unrealistic scaling expectation	deployment failure	model inaccuracy	lab to product gap



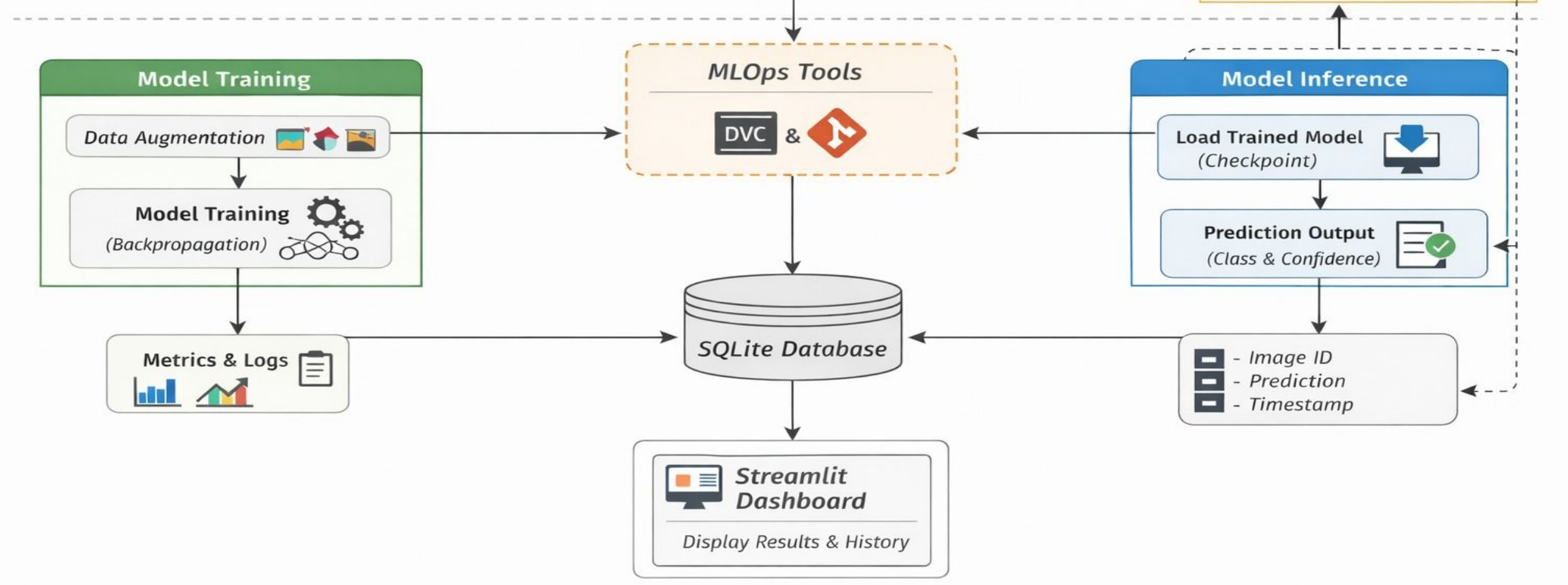
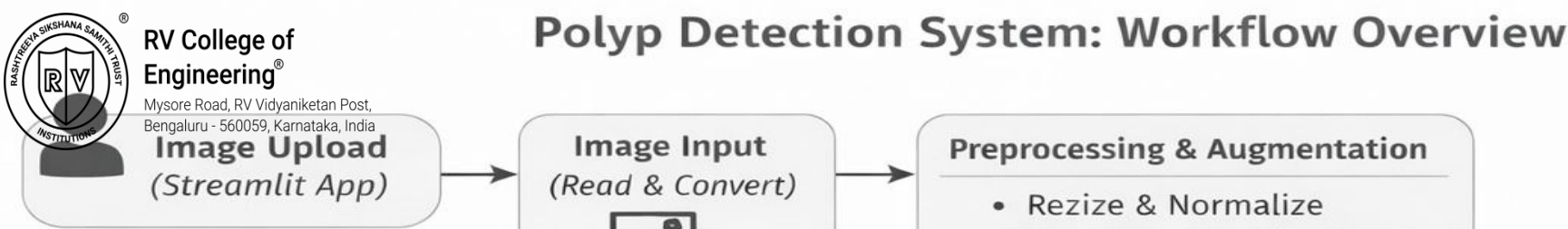
Tools Used

Component	Tool	Purpose
Model Training	PyTorch + Resnet101	CNN training
Experiment Tracking	MLflow	Metrics & runs
Data Versioning	DVC	Dataset & model tracking
Version Control	Git	Code tracking
Drift Detection	Evidently AI	Data drift reports
Monitoring	Grafana	System metrics
Deployment	TorchServe	Model serving
Prediction Logging	Sqlite	Storing predictions



Tools Used







Phase I – ML Life Cycle

- Exploratory Data Analysis
- List of algorithms chosen to build ML model
- Metrics used to evaluate the ML model
- Evaluation of ML model
- Identification of Best Model



Analysis

Exploratory Data Analysis

- Visual inspection of sample images
- Analysis of class distribution
- Examination of image resolutions across the dataset, resizing all images to 224×224 during preprocessing to match ResNet-101 input requirements
- Verified absence of corrupted or unreadable images, ensuring consistent folder-based labeling using **ImageFolder**
- Dataset split into **training (70%)** and **validation (30%)**
- Split performed at file level to avoid data leakage

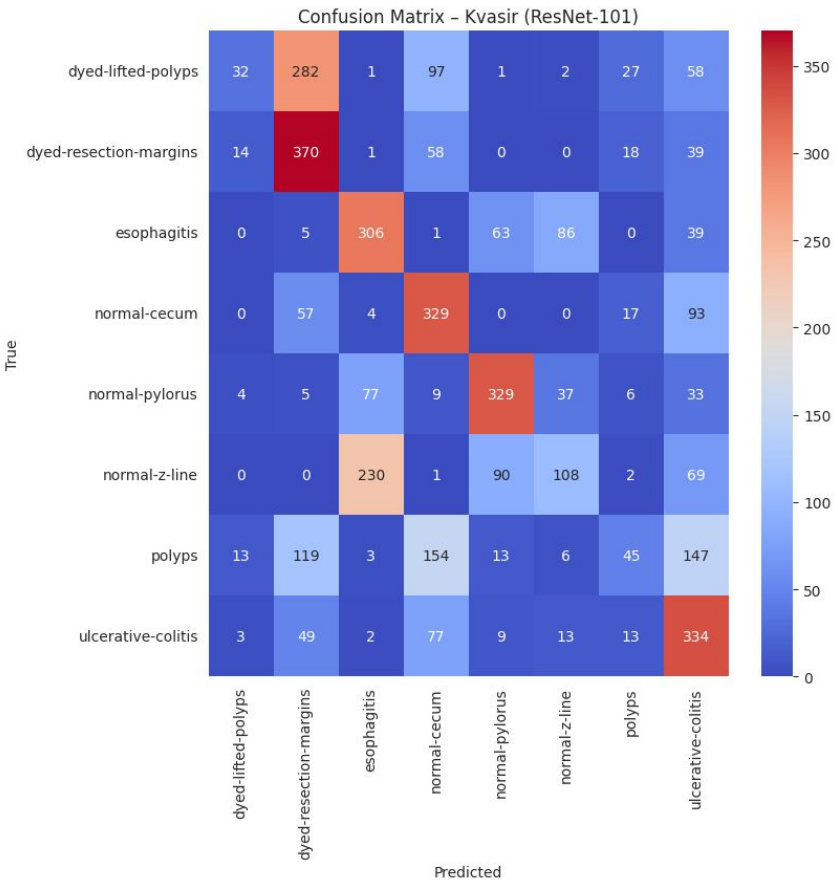
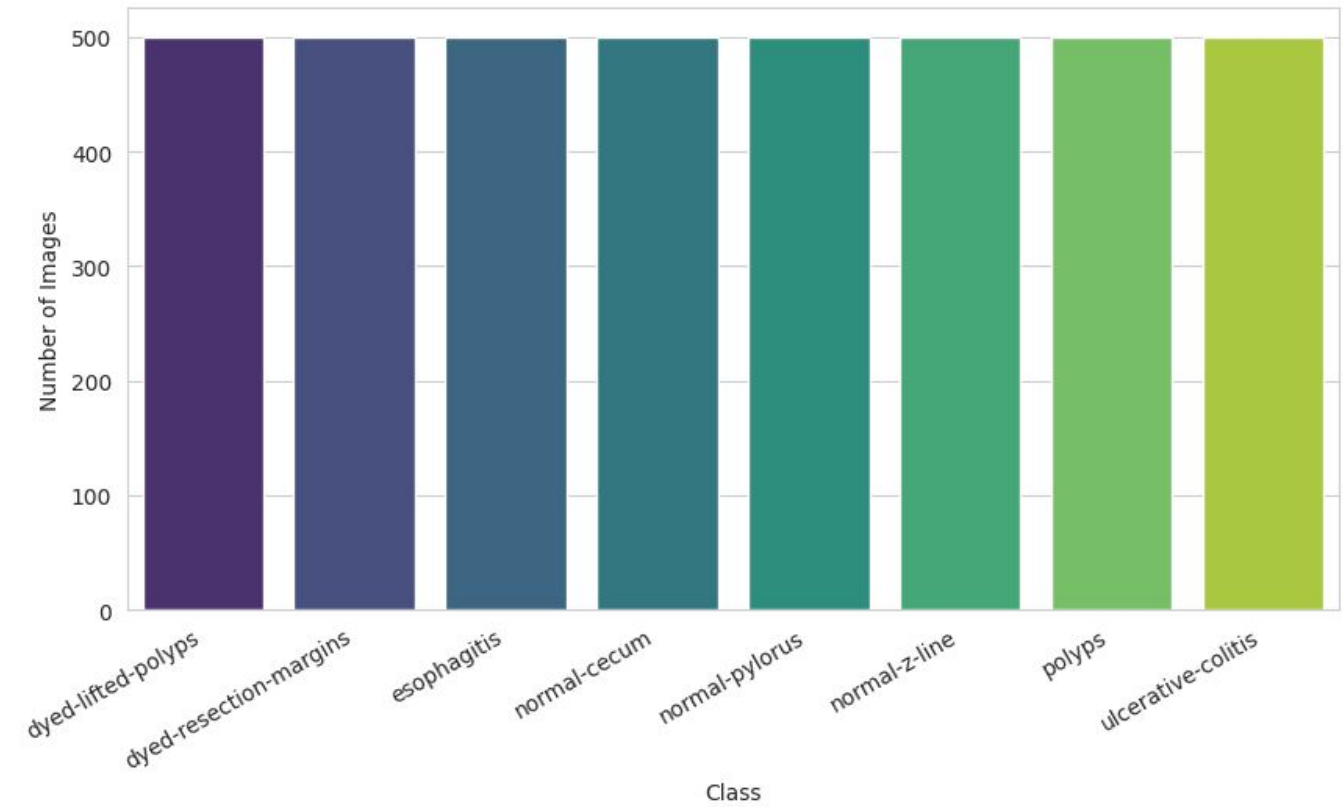
Metrics Used

- Accuracy, Precision, Recall, F1-score, Confusion matrix



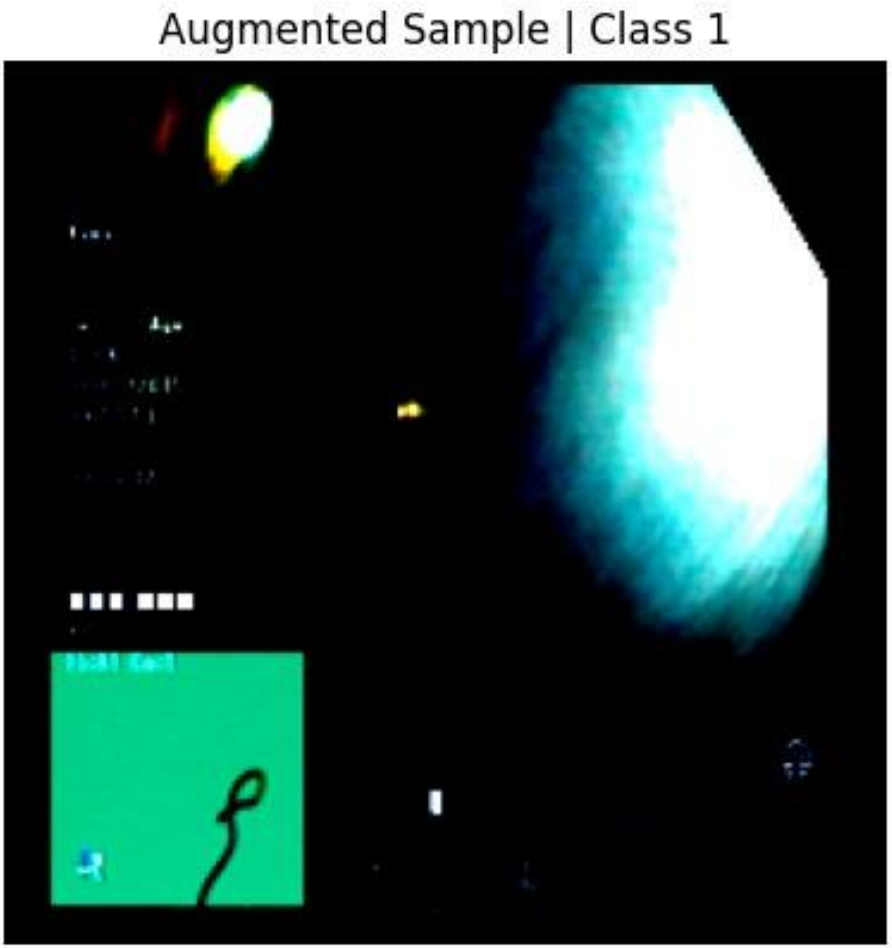
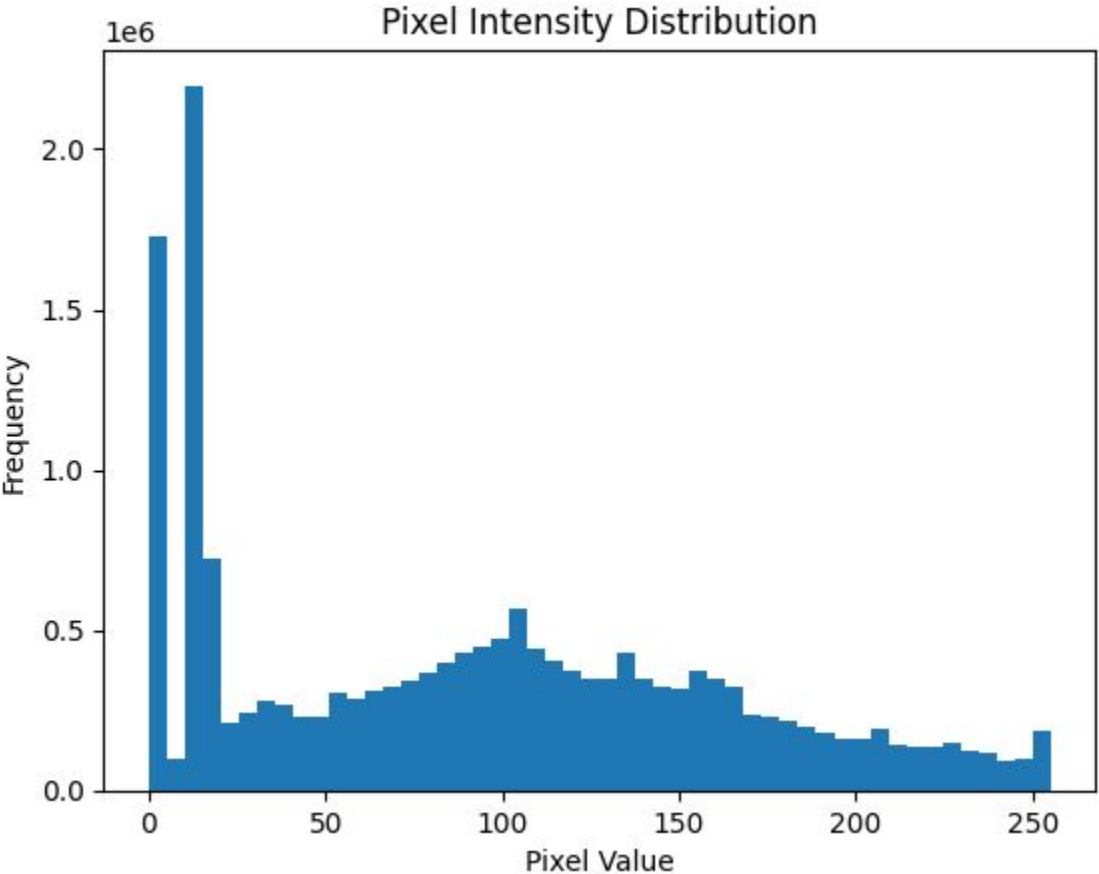
EDA

Kvasir Dataset – Class Distribution





EDA CONTD.

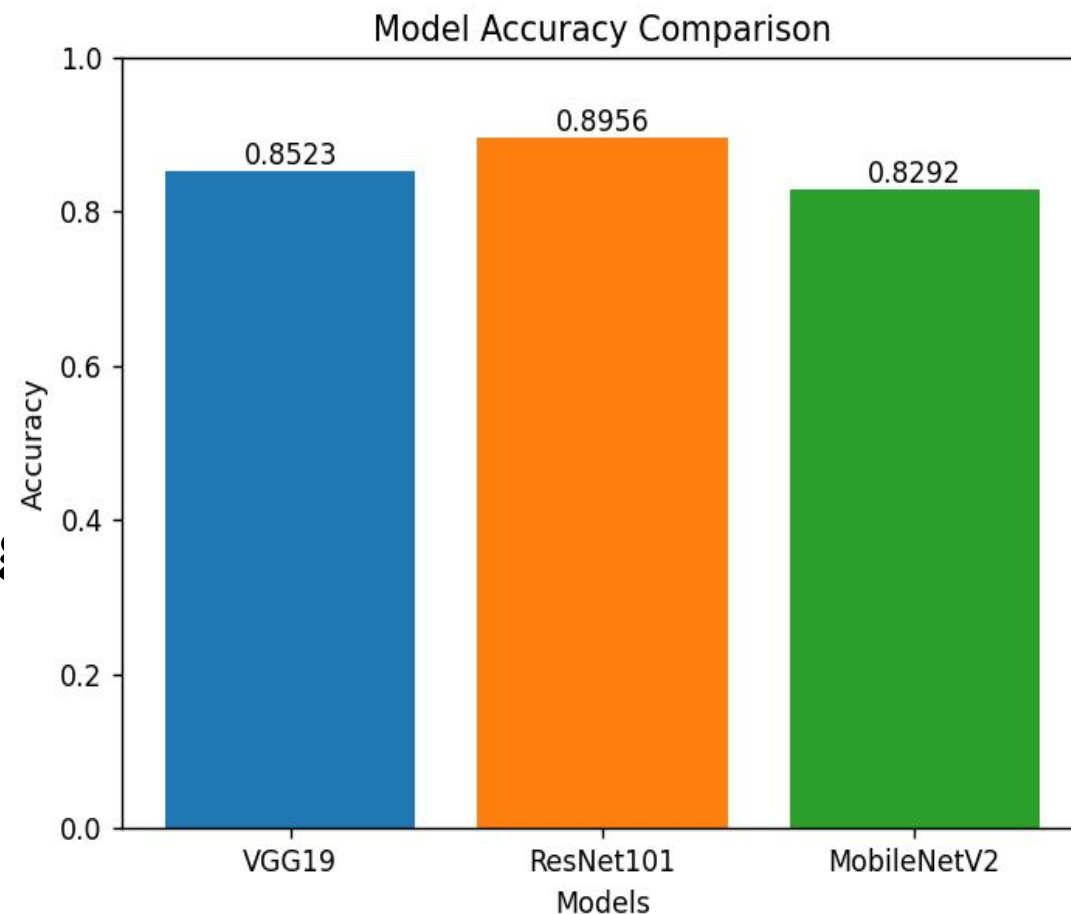


Analysis

Models Used

As mentioned in the previous phases we tried three different models as follows and their accuracies were compared to give the follow result

- Frozen ResNet101 (transfer learning)
(best model)
- VGG-19
- MobileNetV2





Analysis contd.

Parameter	VGG19	ResNet101	MobileNetV2
Accuracy	0.8523	0.8956	0.8292
Precision	~0.85	~0.90	~0.83
Recall	~0.84	~0.89	~0.82
F1-Score	~0.845	~0.895	~0.825
Number of Parameters	~143 M	~44.5 M	~3.5 M
Inference Speed	Slow	Medium	Fast
Training Time	Long	Medium–Long	Short



Analysis

```
Final Metrics:
Confusion Matrix:
[[129  18   0   1   0   0   1   1]
 [ 32 114   0   1   0   0   3   0]
 [  0   1 117   0   0  33   0   0]
 [  0   0   0 143   0   0   6   1]
 [  0   0   1   0 146   0   2   1]
 [  0   1  14   0   1 134   0   0]
 [  1   0   1   3   3   0 134   8]
 [  0   1   0   1   2   0   8 138]]
Precision (weighted): 0.8799
Recall (weighted): 0.8784
F1-Score (weighted): 0.8780
Classification Report:
```

	precision	recall	f1-score	support
dyed-lifted-polyps	0.80	0.86	0.83	150
dyed-resection-margins	0.84	0.76	0.80	150
esophagitis	0.88	0.77	0.82	151
normal-cecum	0.96	0.95	0.96	150
normal-pylorus	0.96	0.97	0.97	150
normal-z-line	0.80	0.89	0.85	150
polyps	0.87	0.89	0.88	150
ulcerative-colitis	0.93	0.92	0.92	150
accuracy			0.88	1201
macro avg	0.88	0.88	0.88	1201
weighted avg	0.88	0.88	0.88	1201



Analysis

```
Anaconda Prompt - python s × + ▾

(polyps) G:\5th_Sem\EL_1\polyps>python src/ml/train.py --data_dir data/raw/Kvasir-dataset/kvasir-dataset-split --epochs 50 --batch_size 16 --lr 1e-4 --output
models/kvasir_resnet101.pth
Splitting dataset...
Dataset split completed.
G:\5th_Sem\EL_1\polyps\src\ml\train.py:104: FutureWarning: `torch.cuda.amp.GradScaler(args...)` is deprecated. Please use `torch.amp.GradScaler('cuda', args
...)` instead.
    scaler = GradScaler()
G:\4th_Sem\EL_1\Anaconda\envs\polyps\lib\site-packages\mlflow\tracking\_tracking_service\utils.py:177: FutureWarning: The filesystem tracking backend (e.g.,
'./mlruns') will be deprecated in February 2026. Consider transitioning to a database backend (e.g., 'sqlite:///mlflow.db') to take advantage of the latest
MLflow features. See https://github.com/mlflow/mlflow/issues/18534 for more details and migration guidance.
    return FileStore(store_uri, store_uri)
2025/12/28 12:14:46 INFO mlflow.system_metrics.system_metrics_monitor: Skip logging GPU metrics. Set logger level to DEBUG for more details.
2025/12/28 12:14:46 INFO mlflow.system_metrics.system_metrics_monitor: Started monitoring system metrics.
Epoch 1/50: 0%| 0/176 [00:00<?, ?it/s]G
:\5th_Sem\EL_1\polyps\src\ml\train.py:124: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)`
instead.
    with autocast():
Epoch 1/50: 100%| 176/176 [01:06<00:00, 2.65it/s]
[1/50] Train Acc: 47.45% | Val Acc: 65.45%
Epoch 2/50: 100%| 176/176 [01:04<00:00, 2.73it/s]
[2/50] Train Acc: 72.15% | Val Acc: 76.27%
Epoch 3/50: 100%| 176/176 [01:02<00:00, 2.83it/s]
[3/50] Train Acc: 76.04% | Val Acc: 80.10%
Epoch 4/50: 100%| 176/176 [01:02<00:00, 2.80it/s]
[4/50] Train Acc: 79.69% | Val Acc: 80.77%
Epoch 5/50: 100%| 176/176 [01:03<00:00, 2.75it/s]
[5/50] Train Acc: 80.69% | Val Acc: 81.35%
Epoch 6/50: 100%| 176/176 [01:06<00:00, 2.67it/s]
[6/50] Train Acc: 80.94% | Val Acc: 82.93%
Epoch 7/50: 0%| 0/176 [00:00<?, ?it/s]
```



Evaluation and Best Model Identification

- Multiple experiments were conducted with different hyperparameters
- Validation performance was used to select the best model
- Final model achieved **high validation accuracy with stable loss**
- Best model was stored and versioned using **DVC and MLflow**



Phase II – MLOPs Life Cycle

	Component	MLOPs Tools
1	Data Management	DVC
2	Experiment Tracking and Version Control	MLflow, Git
3	Automation and Pipeline Orchestration	DVC pipelines
4	Model Deployment & Serving	Torchserve
5	Monitoring & Governance and collaboration	Grafana, Evidently AI, GitHub



Phase II – MLOPs Life Cycle

- Dataset versioned using DVC
- Model training tracked with MLflow
- Model Predictions monitored using Grafana
- Model deployed using TorchServe
- Predictions served via curl http requests
- Drift reports generated using EvidentlyAI



MLFlow

- MLflow is an **experiment tracking tool** used to record machine learning experiments, including model parameters, evaluation metrics, and trained model artifacts.
- MLflow helps by **logging hyperparameters, evaluation metrics, and trained models** for each experiment, enabling easy comparison of multiple load prediction runs.
- This ensures **reproducibility, transparency, and proper model governance** throughout the load prediction pipeline.



MLFlow

- Every ResNet101 training run is logged as a separate experiment.
- We track hyperparameters like learning rate, batch size, and epochs.
- Training and validation metrics are automatically logged for comparison.
- MLflow allows **side-by-side comparison of multiple runs** to detect overfitting.



MLFlow

- All trained models are stored as **artifacts** with their exact configurations.
- The **MLflow Model Registry** manages model versions used for deployment.
- Each registered model is traceable to its dataset version and code version.
- This enables safe rollback if a deployed model underperforms.



MLFlow





MLFlow

mlflow 3.7.0

GitHub Docs

Kvasir-GI-Tract-ResNet101-Frozen Machine learning

Runs Models Traces

metrics.rmse < 1 and params.model = "tree"

Time created State: Active Datasets Sort: val_acc

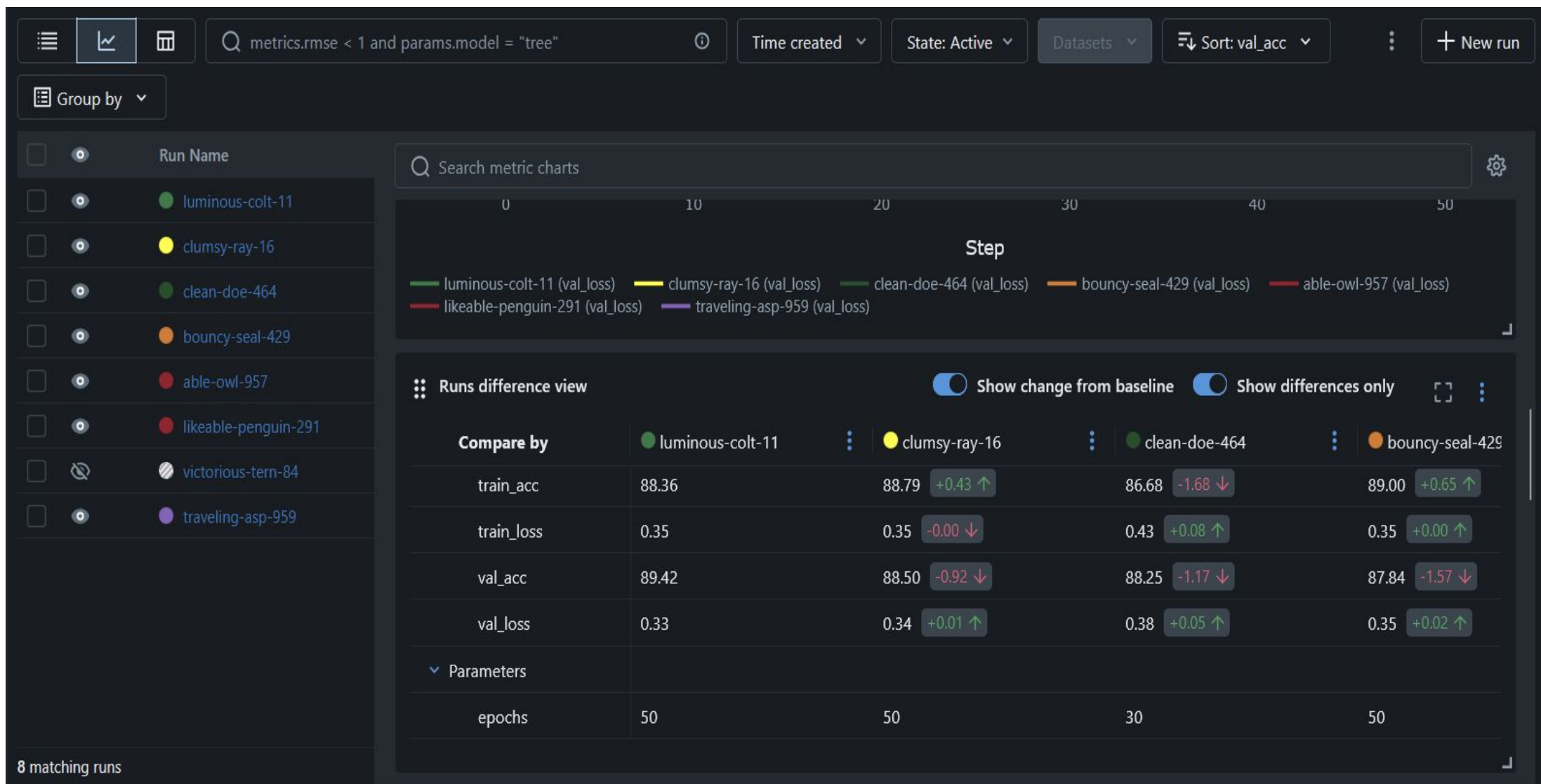
Columns Group by

			Metrics		Parameters					
	Run Name	Created	train_acc	val_acc	architecture	batch_size	data_dir	epochs	img_size	lr
☐	luminous-colt-11	11 hours ago	88.35714285...	89.41666666...	ResNet101-f...	16	data/raw/Kv...	50	224	0.0001
☐	clumsy-ray-16	5 days ago	88.78571428...	88.5	ResNet101-f...	16	data/raw/Kv...	50	224	0.0001
☐	clean-doe-464	4 days ago	86.67857142...	88.25	ResNet101-f...	16	data/raw/Kv...	30	224	0.0001
☐	bouncy-seal-429	9 hours ago	89.00392716...	87.84346378...	ResNet101-f...	16	data/raw/Kv...	50	224	0.0001
☐	able-owl-957	8 hours ago	88.39700107...	87.67693588...	ResNet101-f...	16	data/raw/Kv...	50	224	0.0001
☐	likeable-penguin-291	8 days ago	83.64285714...	85.91666666...	ResNet101-f...	16	data/raw/Kv...	10	224	0.0001
☐	victorious-tern-84	7 days ago	83.82142857...	85.58333333...	ResNet101-f...	16	data/raw/Kv...	10	224	0.0001
☐	traveling-asp-959	7 days ago	83.28571428...	85.5	ResNet101-f...	16	data/raw/Kv...	10	224	0.0001

8 matching runs

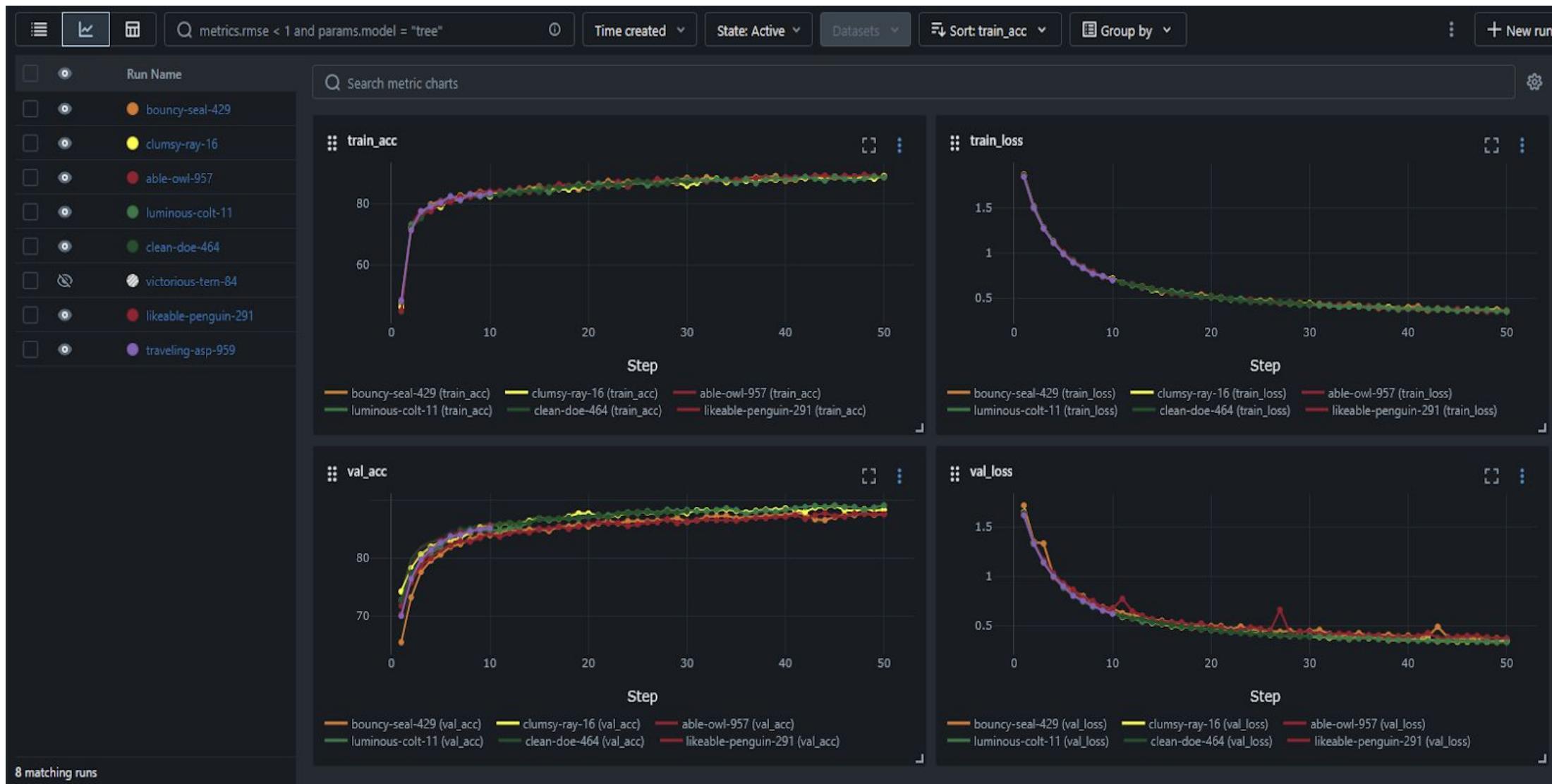


MLFlow





MLFlow





DVC

- DVC is a **data version control tool** used to manage and track large datasets and machine learning artifacts without storing them directly in Git.
- DVC helps by **versioning datasets, intermediate pipeline outputs, and trained model files**, ensuring consistent data usage across experiments.
- This guarantees **reproducibility of model training** and enables **precise tracking of data evolution** throughout the project lifecycle.



DVC

- Each version of the Kvasir dataset is tracked using DVC metadata files.
- **Train-validation splits** are reproducible and linked to specific Git commits.
- Model artifacts such as **trained .pth files** are versioned through DVC pipelines.



DVC

- DVC ensures that every MLflow experiment is associated with the correct dataset version.
- This prevents data leakage and accidental retraining on modified datasets.
- The **DVC pipeline** (**dvc.yaml**) defines the full training workflow from data preparation to model generation.



DVC

```
(polyps) G:\5th_Sem\EL_1\po
lyps>dvc dag
WARNING: Unable to find `le
ss` in the PATH. Check out
<https://man.dvc.org/pipeli
ne/show> for more info.
+-----+
| data\raw.dvc |
+-----+
      *
      *
      *
+-----+
|  train  |
+-----+

(polyps) G:\5th_Sem\EL_1\po
lyps>|
```



DVC

```
Anaconda Prompt (Anaconda) × + ▾  
  
(base) C:\Users\Zaid>G:  
  
(base) G:\>cd 5th_Sem/EL_1/polyps  
  
(base) G:\5th_Sem\EL_1\polyps>conda activate polyps  
  
(polyps) G:\5th_Sem\EL_1\polyps>type data\raw.dvc  
outs:  
- md5: 63e1eb56fd9398c26d85716ff810b261.dir  
  size: 2506150756  
  nfiles: 8004  
  hash: md5  
  path: raw  
  
(polyps) G:\5th_Sem\EL_1\polyps>|
```




DVC

```

Anaconda Prompt - mlflow u x Anaconda Prompt (Anaconda) x Anaconda Prompt (Anaconda) x + v
(polyps) G:\5th_Sem\EL_1\polyps>dvc repro
Verifying data sources in stage: 'data\raw.dvc'

Running stage 'train':
> python src/ml/train.py ^ --data_dir data/raw/Kvasir-dataset/kvasir-dataset-split ^ --epochs 50 ^ --batch_size 16 ^ --lr 1e-4 ^ --output models/kvasir_resnet101.pth
Dataset already split. Skipping.
G:\5th_Sem\EL_1\polyps\src\ml\train.py:104: FutureWarning: 'torch.cuda.amp.GradScaler(args...)' is deprecated. Please use 'torch.amp.GradScaler('cuda', args...)' instead.
    scaler = GradScaler()
G:\4th_Sem\EL_1\Anaconda\envs\polyps\lib\site-packages\mlflow\tracking\_tracking_service\utils.py:177: FutureWarning: The filesystem tracking backend (e.g., './mlruns') will be deprecated
in February 2026. Consider transitioning to a database backend (e.g., 'sqlite:///mlflow.db') to take advantage of the latest MLflow features. See https://github.com/mlflow/mlflow/issues/18
534 for more details and migration guidance.
    return FileStore(store_uri, store_uri)
2025/12/28 13:42:24 INFO mlflow.system_metrics.system_metrics_monitor: Skip logging GPU metrics. Set logger level to DEBUG for more details.
2025/12/28 13:42:24 INFO mlflow.system_metrics.system_metrics_monitor: Started monitoring system metrics.
Epoch 1/50: 0% | 0/176 [00:00<?, ?it/s]G
:\5th_Sem\EL_1\polyps\src\ml\train.py:124: FutureWarning: 'torch.cuda.amp.autocast(args...)' is deprecated. Please use 'torch.amp.autocast('cuda', args...)' instead.
    with autocast():
Epoch 1/50: 100% | 176/176 [01:03<00:00, 2.78it/s]
[1/50] Train Acc: 48.63% | Val Acc: 71.77%
Epoch 2/50: 100% | 176/176 [01:07<00:00, 2.62it/s]
[2/50] Train Acc: 73.26% | Val Acc: 77.44%
Epoch 3/50: 100% | 176/176 [01:11<00:00, 2.46it/s]
[3/50] Train Acc: 77.51% | Val Acc: 80.27%
Epoch 4/50: 100% | 176/176 [01:01<00:00, 2.84it/s]
[4/50] Train Acc: 78.44% | Val Acc: 80.85%
Epoch 5/50: 100% | 176/176 [01:02<00:00, 2.83it/s]
[5/50] Train Acc: 80.72% | Val Acc: 82.35%
Epoch 6/50: 100% | 176/176 [01:00<00:00, 2.90it/s]
[6/50] Train Acc: 81.90% | Val Acc: 82.76%
Epoch 7/50: 100% | 176/176 [01:03<00:00, 2.79it/s]
[7/50] Train Acc: 81.97% | Val Acc: 83.43%
Epoch 8/50: 100% | 176/176 [01:02<00:00, 2.82it/s]
[8/50] Train Acc: 82.58% | Val Acc: 82.85%
Epoch 9/50: 100% | 176/176 [01:00<00:00, 2.91it/s]
[9/50] Train Acc: 83.29% | Val Acc: 84.01%
Epoch 10/50: 100% | 176/176 [01:05<00:00, 2.69it/s]
[10/50] Train Acc: 83.76% | Val Acc: 84.76%
Epoch 11/50: 100% | 176/176 [01:03<00:00, 2.78it/s]
[11/50] Train Acc: 83.93% | Val Acc: 83.35%
Epoch 12/50: 100% | 176/176 [01:01<00:00, 2.87it/s]
[12/50] Train Acc: 83.18% | Val Acc: 84.76%
Epoch 13/50: 100% | 176/176 [01:01<00:00, 2.88it/s]
[13/50] Train Acc: 84.93% | Val Acc: 84.85%
Epoch 14/50: 100% | 176/176 [01:02<00:00, 2.83it/s]
[14/50] Train Acc: 83.65% | Val Acc: 84.35%
Epoch 15/50: 100% | 176/176 [01:03<00:00, 2.79it/s]
[15/50] Train Acc: 85.26% | Val Acc: 85.51%
Epoch 16/50: 100% | 176/176 [01:00<00:00, 2.93it/s]
[16/50] Train Acc: 85.01% | Val Acc: 85.18%
Epoch 17/50: 100% | 176/176 [01:00<00:00, 2.89it/s]

```



Git

- All **source code** (training, inference, TorchServe handlers, drift detection, and monitoring scripts) is tracked using Git.
- Each change to model logic, preprocessing steps, or deployment configuration is committed with a clear history.
- Git enables rollback to a stable version if a new change introduces errors.



Git

- Code versions are tightly coupled with:
 - **Dataset versions (tracked using DVC)**
 - **Model versions (tracked using MLflow)**

This ensures that every trained model can be traced back to:

- Exact code version
- Exact dataset version
- Exact training configuration

Git provides the foundation for future **CI/CD integration** and automated retraining pipelines.



Git

```
(polyps) G:\5th_Sem\EL_1\polyps>dvc push
Collecting |8.04k [00:01, 7.81kentry/s]
Pushing
2 files pushed

(polyps) G:\5th_Sem\EL_1\polyps>git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   data/raw.dvc
        modified:   dvc.lock

(polyps) G:\5th_Sem\EL_1\polyps>git add dvc.lock

(polyps) G:\5th_Sem\EL_1\polyps>git commit -m "Re-trained model"
[master 3cdf048] Re-trained model
2 files changed, 8 insertions(+), 8 deletions(-)

(polyps) G:\5th_Sem\EL_1\polyps>git status
On branch master
nothing to commit, working tree clean

(polyps) G:\5th_Sem\EL_1\polyps>dvc status
Data and pipelines are up to date.
```



Git

```
Anaconda Prompt - mlflow ui X Anaconda Prompt (Anaconda) X Anaconda Prompt (Anaconda) X + v
(polyps) G:\5th_Sem\EL_1\polyps>git log --oneline --graph --all
* 3cdf048 (HEAD -> master) Re-trained model
* 515c763 Rebuild dataset split and retrain model after deletion
* efb8a0 reverted back train_ratio to 0.7
* 826de97 Increased train_ratio to 0.8
* 94ab780 Stop tracking model artifact with Git
* 3a6de9d Rollback to previous version
* ea03101 model added
* 7592f96 Updated requirements.txt
* 406b2c3 Removed tensorflow package from requirements.txt
* 21387a9 zaid
* 7e4eb60 Added Torchserve runtime directories to gitignore file
* 54d02f2 Edited configuration file to add torchserve endpoints and disable token authentication
* 0f028a6 Removed redundant routes
* ddce3bd Updated Torchserve deployment logic
* ab34714 Updated instructions file
* 9bfb1d5 Updated config file
* 82bbf60 Updated config file
* f8274fb Updated torchserve handler file
* 03e5f3a Added configuration file to disable token authentication for torchserve
* 746722f Updated instructions file
* 37abe22 Updated instructions file
* 30b6865 Updated torchserve code
* d8d14e9 Add TorchServe serving module
* bd2a8da Updated frontend code for single image upload + prediction
* fc9f438 Reproduce training with final metrics
* bc5a87f Changed epochs to 30
* b032c12 Add final validation metrics output
* 3f72c6a Updated training pipeline metrics
* 8293109 Reproduce training pipeline
* 46bd954 Updated streamlit app code
* d27438d Commented out Model Re-training Function in streamlit file
* ddb11c9 stopped tracking drift results
* dcd761d Updated drift monitoring file
* ad99a0a Updated drift monitoring file
* 9d2bac5 Increase epochs to 50 for ResNet101 training
* 795b460 Added Drift monitoring file
* 7ef113d Incomplete drift monitoring file
* decc74f Removed duplicate db file
* 07cf758 Stop tracking runtime SQLite prediction database
* 61ffddb Updated gitignore file to ignore runtime db
```



Grafana

- Grafana is a **visualization and monitoring tool** used to display system and model metrics in real time.
- Grafana helps by providing dashboards to track **inference behavior, performance trends**, etc..
- This enables **continuous observability** of deployed machine learning models.

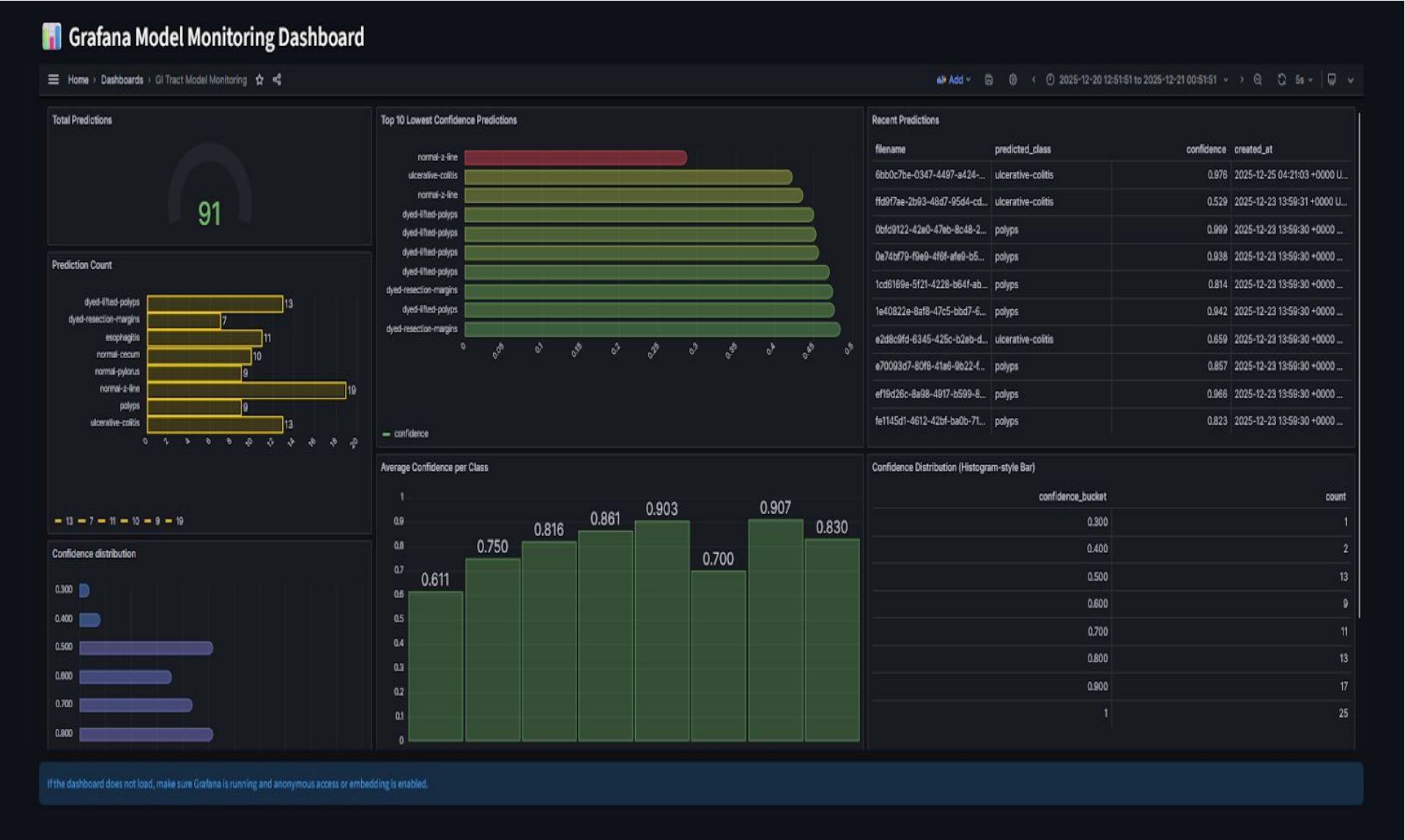


Grafana

- **Prediction confidence metrics** generated from predicted outputs are visualized.
- Time-based trends allow **early identification of model instability**.
- Grafana provides **centralized monitoring for model performance validation**.



Grafana





Torchserve

- TorchServe is a **model serving framework** used to deploy trained PyTorch models as scalable REST API endpoints.
- TorchServe helps by **packaging trained models into deployable .mar files** and exposing inference APIs for real-time predictions.
- This enables **seamless local deployment** and **testing of trained models** without modifying training code.



Torchserve

- The trained ResNet101 model is archived using **torch-model-archiver**.
- A custom inference handler processes incoming images, applies preprocessing, and runs model inference.
- TorchServe exposes prediction endpoints via **HTTP (/predictions/gi)**.



Torchserve

- The model runs independently from the training environment in a separate serving process.
- This simulates real-world production deployment scenarios.



Torchserve

```
Anaconda Prompt - mlflow ui X Anaconda Prompt (Anaconda) X Anaconda Prompt (Anaconda) X + v

(polyyps) G:\5th_Sem\EL_1\polyyps>curl http://127.0.0.1:8080/ping
{
  "status": "Healthy"
}

(polyyps) G:\5th_Sem\EL_1\polyyps>curl http://127.0.0.1:8081/models
{
  "models": [
    {
      "modelName": "gi",
      "modelUrl": "gi.mar"
    }
  ]
}

(polyyps) G:\5th_Sem\EL_1\polyyps>curl -X POST http://127.0.0.1:8080/predictions/gi ^ -H "Content-Type: application/octet-stream" ^ --data-binary "@G:\5th_Sem\EL_1\polyyps\data\raw\Kvasir-dataset\kvasir-dataset-split\val\polyyps\0a994cfe-761c-4d53-ad76-668927ccd347.jpg"
{
  "class": "polyyps",
  "confidence": 0.9739460349082947
}

(polyyps) G:\5th_Sem\EL_1\polyyps>|
```



Evidently AI

- Evidently AI is a **model monitoring and drift detection tool** used to analyze changes in data and model behavior after deployment.
- Evidently helps by **detecting data drift** and **prediction confidence drift** between reference and current datasets.
- This ensures **model reliability** and **alerts developers** to potential degradation in model performance.



Evidently AI

- **Baseline (training) predictions** are used as reference data.
- **Inference results** are compared against the reference distribution.
- Statistical tests such as the **Kolmogorov–Smirnov test** are applied for drift detection.



Evidently AI

- **Confidence score distributions** are monitored to detect model uncertainty shifts.
- **Interactive HTML reports** provide detailed drift visualizations.
- This enables **informed decisions on model retraining or redeployment**.



Evidently AI





Conclusion

- Successfully implemented an **end-to-end MLOps pipeline**
- Ensured reproducibility, monitoring, and deployment
- Demonstrated real-world ML system lifecycle
- Project aligns with **industry MLOps best practices**